

Aula 07 — RAG: Retrieval-Augmented Generation

SGAIM 2025/2026 — Semana 7

Contexto

Na semana 6 demos ao LLM **mãos** — structured output e tool calling. Hoje resolvemos outra limitação fundamental: o LLM não tem acesso a conhecimento que não está nos seus pesos. Alucina porque optimiza verosimilhança, não verdade. RAG é a solução: injectar documentos relevantes no momento da pergunta.

Vão construir um pipeline RAG completo em Python puro — sem frameworks. O objectivo é **compreender cada passo**, não usar uma biblioteca que esconde os detalhes.

Método: antes de cada exercício, escrevam numa frase o que esperam que aconteça. Depois testem. Confirmou-se?

Pré-requisitos

Software

```
pip install sentence-transformers chromadb requests
```

Ollama já instalado (semana 05) com pelo menos llama3.2:3b.

Corpus

O corpus para esta aula é o **CDRAM** — 12 documentos fictícios sobre o *Clube Desportivo e Recreativo Atlético Monsanto*. Está disponível no Moodle (pasta cdram-corpus/). Inclui regulamentos, calendário, horários, inscrições, código de conduta, actas e histórico do clube.

Porquê um corpus inventado? Um pipeline RAG só é interessante se o LLM **precisar** do corpus para responder. Se usássemos documentação pública (Wikipedia, docs da OpenAI), o LLM já conhece as respostas pelos pesos — e nunca saberíamos se o retrieval está a funcionar ou se ele está apenas a responder “de cabeça”. O CDRAM não existe: o LLM **tem** de ler os documentos. É um sandbox controlado para a aula. No MP2 vão escolher o vosso próprio corpus real — com o mesmo critério: fora dos pesos.

Exercício 1 — Ingestão e chunking simples (~15 min)

O primeiro passo de qualquer pipeline RAG é carregar documentos e dividi-los em pedaços (chunks).

1.1 — Carregar o corpus

Criar um ficheiro rag_pipeline.py e começar:

```
import os

CORPUS_DIR = "cdram-corpus/"

def load_corpus(directory):
    """Carrega todos os ficheiros .md do diretório."""
    documents = []
    for filename in os.listdir(directory):
        if filename.endswith(".md"):
            with open(os.path.join(directory, filename), "r", encoding="utf-8") as f:
                documents.append({
                    "content": f.read(),
                    "source": filename
                })
    return documents
```

```
docs = load_corpus(CORPUS_DIR)
print(f"Carregados {len(docs)} documentos")
for d in docs:
    print(f"    {d['source']}: {len(d['content'])} caracteres")
```

Pergunta: Quantos caracteres tem o documento mais longo? E o mais curto? Porque é que isto importa para o chunking?

1.2 — Chunking fixed-size

Implementar chunking por número fixo de caracteres, com overlap:

```
def chunk_fixed_size(text, chunk_size=500, overlap=50):
    """Divide o texto em chunks de tamanho fixo com overlap."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start += chunk_size - overlap
    return chunks
```

Testar:

```
for doc in docs[:2]: # só os 2 primeiros para testar
    chunks = chunk_fixed_size(doc["content"])
    print(f"\n{doc['source']}: {len(chunks)} chunks")
    print(f"    Primeiro chunk: {chunks[0][:100]}...")
    print(f"    Último chunk: {chunks[-1][:100]}...")
```

Pergunta: O que acontece se um chunk cair no meio de uma frase? E no meio de uma palavra? Que informação se perde?

Exercício 2 — Embeddings (~15 min)

Agora vamos transformar cada chunk num vector.

2.1 — Carregar o modelo de embeddings

```
from sentence_transformers import SentenceTransformer

# Encoder multilíngue (50+ línguas, inclui PT) — 384 dimensões.
model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
print(f"Modelo carregado. Dimensão: {model.get_sentence_embedding_dimension()}")
```

Nota: O download do modelo (~470 MB) acontece na primeira execução. Se demorar, trabalhem em par com quem já tenha descarregado. Se a rede engasgar, correr este bloco em casa antes da próxima aula.

Porquê um modelo multilíngue? O corpus é em português. Modelos treinados só em inglês (ex. all-MiniLM-L6-v2) dão retrieval mais fraco em textos PT — especialmente quando a query usa vocabulário comum e o documento usa vocabulário técnico. Regra prática: **alinhar a língua do encoder com a língua do corpus.**

2.2 — Embedder os chunks

```
all_chunks = []
for doc in docs:
    chunks = chunk_fixed_size(doc["content"])
    for i, chunk in enumerate(chunks):
        all_chunks.append({
```

```

        "text": chunk,
        "source": doc["source"],
        "chunk_index": i
    })

print(f"Total de chunks: {len(all_chunks)}")

# Embedder todos os chunks de uma vez (eficiente)
embeddings = model.encode([c["text"] for c in all_chunks], show_progress_bar=True)
print(f"Shape: {embeddings.shape}") # (n_chunks, 384)

```

Pergunta: Porque é que fazemos batch encoding em vez de embedder um chunk de cada vez? O que é que o shape nos diz?

2.3 — Verificar similaridade

```

import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Embedder uma query
query = "Qual é o valor da quota mensal de um sócio efectivo?"
query_embedding = model.encode([query])[0]

# Comparar com os primeiros 5 chunks
for i in range(5):
    sim = cosine_similarity(query_embedding, embeddings[i])
    print(f"Chunk {i} ({all_chunks[i]['source'][:20]}...): sim = {sim:.3f}")
    print(f"  Texto: {all_chunks[i]['text'][:80]}...")

```

Pergunta: Os chunks com similaridade mais alta fazem sentido? São relevantes para a query?

Exercício 3 — Armazenar no ChromaDB (~10 min)

3.1 — Criar a coleção

```

import chromadb

client = chromadb.Client() # in-memory, sem persistência
collection = client.create_collection(
    name="cdram_corpus",
    metadata={"description": "Documentos do CDRAM (corpus da aula 07)"}
)

# Adicionar os chunks
collection.add(
    ids=[f"chunk_{i}" for i in range(len(all_chunks))],
    documents=[c["text"] for c in all_chunks],
    metadatas=[{"source": c["source"], "chunk_index": c["chunk_index"]} for c in all_chunks],
    embeddings=embeddings.tolist()
)

print(f"Coleção criada com {collection.count()} documentos")

```

Nota: O ChromaDB pode também calcular embeddings automaticamente se não fornecermos os nossos. Mas estamos a usar os nossos embeddings para ter controlo total sobre o processo.

3.2 — Fazer uma query

```
results = collection.query(
    query_embeddings=model.encode(["Quais são as modalidades desportivas do clube?"]).tolist(),
    n_results=3
)

for i, (doc, meta, dist) in enumerate(zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0]
)):
    print(f"\n--- Resultado {i+1} (distância: {dist:.3f}) ---")
    print(f"Fonte: {meta['source']}, chunk {meta['chunk_index']}")
    print(f"Texto: {doc[:200]}...")
```

Pergunta: A distância no ChromaDB é similaridade ou distância? O que é que um valor baixo significa? E um valor alto?

Exercício 4 — Retrieval + Geração (~15 min)

Agora juntamos tudo: recuperar chunks relevantes e passar ao LLM para gerar a resposta.

4.1 — Prompt com contexto

```
import requests
import json

OLLAMA_URL = "http://localhost:11434"

SYSTEM_PROMPT = """You are a helpful assistant. Answer the user's question using ONLY the context provided below.
If the context does not contain enough information, say "I don't have enough information to answer."
Do not use your own knowledge. Be concise.

Context:
{context}"""

def generate_with_context(query, retrieved_chunks, model_name="llama3.2:3b"):
    """Gera resposta usando chunks recuperados como contexto."""
    context = "\n\n---\n\n".join(retrieved_chunks)

    response = requests.post(
        f"{OLLAMA_URL}/v1/chat/completions",
        json={
            "model": model_name,
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT.format(context=context)},
                {"role": "user", "content": query},
            ],
            "temperature": 0,
        },
        timeout=60,
    )
    response.raise_for_status()
    return response.json()["choices"][0]["message"]["content"]
```

Nota: Usamos /v1/chat/completions com mensagens separadas (system + user) em vez de /api/generate com um prompt único. Isto separa as instruções do sistema do conteúdo do utilizador — boa prática que devem manter no MP2.

4.2 — Pipeline completo

```
def rag_query(query, collection, embedding_model, llm_model="llama3.2:3b", k=3):
    """Pipeline RAG completo: query → embed → retrieve → generate."""

    # 1. Embedder a query
    query_embedding = embedding_model.encode([query])[0]

    # 2. Recuperar os k chunks mais relevantes
    results = collection.query(
        query_embeddings=query_embedding.tolist(),
        n_results=k
    )

    # 3. Gerar resposta com contexto
    retrieved_chunks = results["documents"][0]
    answer = generate_with_context(query, retrieved_chunks, llm_model)

    return {
        "query": query,
        "retrieved_chunks": retrieved_chunks,
        "sources": results["metadatas"][0],
        "answer": answer
    }
```

4.3 — Testar

```
result = rag_query("Como pode um atleta inscrever-se na Secção de Futsal?", collection, model)

print(f"Query: {result['query']}")
print(f"\nFontes recuperadas:")
for src in result["sources"]:
    print(f" - {src['source']}, chunk {src['chunk_index']}")
print(f"\nResposta: {result['answer']}")
```

Pergunta: A resposta está correcta? Está ancorada nos documentos? Há alucinação? Testem também:

```
# Perguntas para testar
rag_query("A que horas treina o grupo de iniciação à nataç o?", collection, model)
rag_query("Em que data   a Festa de fim de  poca?", collection, model)
rag_query("Qual   o pre o dos bilhetes para o Benfica-Sporting?", collection, model) # off-topic
```

Exerc cio 5 — Experimentar chunking (~15 min)

Agora que o pipeline funciona, vamos ver o impacto do chunking.

5.1 — Chunking por frases

Implementar uma estrat gia alternativa que respeita fronteiras de frases:

```
import re

def chunk_by_sentences(text, max_chunk_size=500, overlap_sentences=1):
    """Divide o texto em chunks que respeitam fronteiras de frases."""
    sentences = re.split(r'(?<=[.!?])\s+', text)
    chunks = []
    current_chunk = []
    current_size = 0

    for sentence in sentences:
        if current_size + len(sentence) > max_chunk_size and current_chunk:
```

```

        chunks.append(" ".join(current_chunk))
        # Overlap: manter as últimas N frases
        current_chunk = current_chunk[-overlap_sentences:]
        current_size = sum(len(s) for s in current_chunk)

    current_chunk.append(sentence)
    current_size += len(sentence)

    if current_chunk:
        chunks.append(" ".join(current_chunk))

    return chunks

```

5.2 — Comparar

Reconstruir o pipeline com chunking por frases e comparar as mesmas queries:

```

# Reconstruir com novo chunking
all_chunks_sent = []
for doc in docs:
    chunks = chunk_by_sentences(doc["content"])
    for i, chunk in enumerate(chunks):
        all_chunks_sent.append({
            "text": chunk,
            "source": doc["source"],
            "chunk_index": i
        })

# (Repetir: embedder, criar nova coleção, fazer as mesmas queries)
# Comparar os resultados com o chunking fixed-size

```

Perguntas: - A mesma query devolve chunks diferentes? Melhores ou piores? - O tamanho dos chunks é mais uniforme ou mais variável? - Em que tipo de pergunta é que cada estratégia é melhor?

Exercício 6 — Observar modos de falha (~10 min)

Testar deliberadamente situações onde o RAG falha:

```

# 1. Query ambígua – vários documentos podem ser relevantes
rag_query("Quais são os horários?", collection, model)
# Horários de treino? Horários da secretaria? Horários de que modalidade?

# 2. Query sobre informação que NÃO está no corpus (armadilha pura)
rag_query("Qual é a modalidade com mais sócios no CDRAM?", collection, model)
# Esta informação nunca é referida em lado nenhum. O LLM admite, ou inventa?

# 3. Termos técnicos vs termos comuns – mesma pergunta, léxico diferente
rag_query("Qual a idade mínima para jogar futsal?", collection, model)
rag_query("Uma criança de 9 anos pode entrar na equipa de futsal?", collection, model)
# As duas queries apontam para o mesmo facto. Os retrievals são iguais?

# 4. Query com contradição deliberada no corpus
rag_query("Qual é o valor actual da quota de um sócio efectivo?", collection, model)
# A tabela de quotas diz 15 €; a acta de Fevereiro de 2025 actualiza para 20 €.
# Qual resposta o RAG dá? Cita a fonte? Reconhece a inconsistência?

```

Registar: Para cada falha, classifiquem:

- O problema está no **retrieval** (não encontrou os chunks certos) ou na **geração** (encontrou mas não usou bem)?
- Mudar a estratégia de chunking ajudaria?

Semente para o MP2: no projecto vão ter de construir um benchmark de 10 perguntas (5 fáceis + 3 médias + 2

armadilhas — perguntas cuja resposta *não existe* no vosso corpus). As perguntas-armadilha são as mais reveladoras: se o sistema responde com confiança a algo que não lá está, está a alucinar. Notem que a query 2 acima é exactamente uma armadilha no corpus CDRAM. Comecem a pensar desde já: que 2 perguntas-armadilha farão ao vosso corpus?

Para terminar

No final da aula vamos partilhar observações. Preparem:

- Qual foi o modo de falha mais interessante que observaram?
- Chunking por frases foi melhor ou pior do que fixed-size? Para que tipo de queries?
- O RAG “acorda” o LLM ou pode enganá-lo com contexto irrelevante?

TPC: - Completar o pipeline com pelo menos 2 estratégias de chunking - Comparar: mesma pergunta, chunks diferentes → resposta diferente? - Registrar observações: onde o RAG falha e porque - **Desafio extra:** trocar para um modelo só EN (ex: all-MiniLM-L6-v2, 80 MB) e correr as mesmas queries. Quais resultados degradam e porquê? Que tipo de queries aguenta, quais não?

O TPC é um ensaio para o MP2. No projecto farão o mesmo exercício — duas estratégias de chunking + registo de falhas — mas sobre o **vosso** corpus e com método formal (predict → measure → explain). O que aprenderem aqui transfere directamente.

Na semana 8: RAG — avaliação e estratégias avançadas. De “funciona” a “funciona bem”.

Ponte para o MP2 — o que muda na pipeline final

A pipeline que construíram hoje tem todas as peças. Para o MP2, há três ajustes operacionais que convém fazer já — não são conceitos novos, é cola para não tropeçarem depois.

1. Embeddings via Ollama (em vez de sentence-transformers)

No MP2 recomendamos nomic-embed-text via Ollama. Porquê? Um único runtime (tudo Ollama, sem bibliotecas extra a descarregar modelos) e embeddings com 768 dimensões vs 384 do MiniLM que usámos hoje. Também é multilíngue — importante se o corpus for PT.

Setup:

```
ollama pull nomic-embed-text
```

Substituir a função de embedding:

```
import requests

OLLAMA_URL = "http://localhost:11434"

def embed_ollama(texts, model="nomic-embed-text"):
    """Equivalente a model.encode(texts) - retorna List[List[float]]."""
    vectors = []
    for t in texts:
        r = requests.post(
            f"{OLLAMA_URL}/api/embeddings",
            json={"model": model, "prompt": t},
            timeout=30,
        )
        r.raise_for_status()
        vectors.append(r.json()["embedding"])
    return vectors
```

Onde antes tinham `model.encode(...)`, passam a ter `embed_ollama(...)`. O resto do pipeline não muda.

Observação: comparar paraphrase-multilingual-MiniLM-L12-v2 vs nomic-embed-text é uma das comparações à escolha no MP2. Que resultados esperam **antes** de medir?

2. Persistência do ChromaDB

O `chromadb.Client()` do guião é **in-memory** — perdem tudo ao fechar o Python. Para o MP2, usem persistência:

```
client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection("corpus_mp2")
```

Embedam uma vez, reutilizam em execuções futuras. Sem isto, cada experiência começa por re-embedar o corpus inteiro.

3. Corpus em PDF

Se o vosso corpus for PDF (legislação, papers, manuais), precisam de extrair texto:

```
pip install pypdf
```

```
from pypdf import PdfReader

def load_pdf(path):
    reader = PdfReader(path)
    return "\n\n".join(
        page.extract_text() for page in reader.pages if page.extract_text()
    )
```

Aviso: PDFs com tabelas, colunas ou imagens produzem texto sujo. **Comecem com markdown/texto simples** e só passem a PDF quando o pipeline estiver estável. O tempo que gastam a limpar texto de PDF é tempo que tiram à análise — e a análise vale 50% da nota.

4. Stack completa recomendada

```
ollama pull llama3.2:3b          # geração (mínimo)
ollama pull qwen2.5:7b          # geração (para comparação 3B vs 7B)
ollama pull nomic-embed-text    # embeddings (recomendado)
ollama pull all-minilm          # embeddings (alternativa para comparação)
```

Se a rede do ISEL for lenta, façam `ollama pull` em casa antes da próxima aula.

Mapa: da aula para o MP2

Cada exercício desta aula tem um correspondente directo no projecto. Vejam-no como um *rehearsal* — se chegarem ao fim com o Exercício 6 a correr, têm a parte técnica do MP2 essencialmente feita.

Exercício da aula 07	Componente do MP2
1 — Ingestão + chunking fixed-size	Ingestão do vosso corpus + comparação de chunking (obrigatória)
2 — Embeddings + similaridade	Pipeline de embedding (sentence-transformers → nomic-embed-text)
3 — ChromaDB (in-memory)	ChromaDB persistente (PersistentClient)
4 — Retrieval + geração	Pipeline completo + prompt de geração
5 — Comparar chunking (TPC)	Comparação obrigatória + benchmark de 10 perguntas
6 — Modos de falha	<i>Failure analysis</i> na 1 página de análise
—	Novo no MP2: corpus próprio + benchmark (5 fáceis + 3 médias + 2 armadilhas) + 1 página de decisões/falhas

Reler desta perspectiva: o guião é o **como**, o MP2 é o **porquê** e o **sobre quê**. A implementação é a mesma; o que muda é a exigência de escolher, justificar e avaliar.